Implantação PortfolioHUB + IA GEMINI (v2)

Documentação Técnica de Engenharia e Relatório de Entrega Final

Estudante: Arthur Bradley Barroca
Curso: Análise e Desenvolvimento de Sistemas (ADS)
Data de Entrega: 14/06/2026
Referência Base: Desafio - Entrega Final.pdf

Apresentação do Estudante à IA Gemini

Olá, Gemini. Meu nome é **Arthur Bradley Barroca**, sou estudante do curso superior de tecnologia em **Análise e Desenvolvimento de Sistemas (ADS)**. Possuo uma afinidade de longa data com inovação tecnológica, arquitetura de computadores e lógica de software, elementos que moldaram minha trajetória e escolha profissional. Meu foco atual de estudos está centrado no desenvolvimento de software estruturado, engenharia de requisitos e na aplicação prática de automações inteligentes (como RPA) para otimização de fluxos operacionais.

Este documento representa o meu relatório consolidado para o desafio de implantação da plataforma **PortfolioHUB**. Utilizo esta trilha documental para registrar como configurei você, IA Gemini, como minha ferramenta de suporte de engenharia, garantindo que cada tomada de decisão — desde a modelagem de diretórios até o endurecimento (*hardening*) de políticas de segurança no GitHub — seguisse estritamente as melhores práticas de desenvolvimento corporativo.

Sumário Analítico

1. Planejamento da Implantação
- 1.1. Criação do Plano de Implantação Detalhado
- 1.2. Configuração do Google GEMINI como Guia Metodológico

2. Configuração Inicial e Integração com GitHub
- 2.1. Criação e Configuração do Ambiente Dedicado
- 2.2. Integração e Armazenamento Dinâmico de Projetos

3. Gestão de Usuários e Segurança
- 3.1. Integração do Controle de Usuários com o GitHub
- 3.2. Implementação de Políticas de Segurança DevSecOps

4. Compartilhamento e Controle de Acesso com GitHub

- 4.1. Controle de Versão e Integração do Repositório
- 4.2. Documentação de Processos e Práticas de Colaboração

5. Finalização da Integração e Testes

- 5.1. Conclusão das Etapas de Integração via API
- 5.2. Preparação do Ambiente de Produção e Lançamento

6. Revisão Final e Apresentação

- 6.1. Estruturação do Roteiro e Apresentação para YouTube

1. Planejamento da Implantação

> **Resumo e Contextualização Técnica:** Esta fase inicial consiste na definição do cronograma de desenvolvimento dividido em 5 fases e na calibração analítica da inteligência artificial por meio de um *System Prompt* especializado. Esta ação serve para mitigar riscos de desvio de escopo e organizar o pipeline de entrega. Configurar o Gemini adequadamente garante que todas as sugestões de código dadas pela IA ao longo do projeto sigam arquiteturas limpas (*Clean Code*) e padrões rigorosos de segurança de mercado, em vez de respostas superficiais ou genéricas.

1.1. Criação do Plano de Implantação Detalhado

> **PROMPT DO ESTUDANTE:**
>
> *"Gemini, preciso iniciar o meu dever final de ADS. O primeiro requisito exige a criação de um plano de implantação detalhado para a plataforma PortfolioHUB. Como posso estruturar as fases desse deploy de forma técnica e organizada?"*

RESPOSTA DO GEMINI:

Para estruturar a implantação do **PortfolioHUB** com rigor técnico para o curso de ADS, dividiremos o ciclo de vida do projeto em 5 fases sequenciais dentro do pipeline de entrega:

- **Fase 1: Setup & Workspaces:** Definição da arquitetura de diretórios, inicialização do repositório Git local e isolamento das chaves de configuração.
- **Fase 2: Conexão com API Externa:** Estabelecimento de requisições assíncronas para a API REST do GitHub para consumo de metadados dos repositórios.
- **Fase 3: Hardening e Governança:** Aplicação de regras de proteção de branch, bloqueio de pushes diretos e varredura automatizada de segurança.
- **Fase 4: Garantia de Qualidade (QA):** Testes funcionais de rotas, verificação de degradação suave de falhas de rede e auditoria de performance.
- **Fase 5: Continuous Deployment (CD):** Deploy automatizado do build de produção em plataforma Cloud (Vercel/Netlify) integrado ao gatilho de merge.

1.2. Configuração do Google GEMINI como Guia Metodológico

> **PROMPT DO ESTUDANTE:**
>
> *"Como devo configurar e instruir você (Google GEMINI) para atuar como meu guia técnico durante toda a trilha de desenvolvimento, garantindo conformidade com os critérios de avaliação?"*

RESPOSTA DO GEMINI:

Para garantir que eu atue estritamente sob os padrões de engenharia de software requeridos, utilize o seguinte prompt de sistema no início do seu workspace de IA:

```
Atue como um Engenheiro de Software Sênior e Especialista em Segurança DevSecOps. Sua função é guiar a implantação do PortfolioHUB integrado ao ecossistema do GitHub. Todas as suas instruções de código devem seguir boas práticas de arquitetura (Clean Code), separação de conceitos, tratamento estrito de exceções e o princípio do menor privilégio em segurança. Não forneça soluções genéricas; use padrões de mercado compatíveis com o nível acadêmico de ADS.
```

2. Configuração Inicial e Integração com GitHub

> **Resumo e Contextualização Técnica:** Aqui ocorre a inicialização física do repositório remoto no GitHub, a montagem de uma arquitetura modular de diretórios (separando assets, componentes isolados, utilitários e serviços) e o consumo de dados da API REST externa via JavaScript assíncrono. O objetivo é criar a fundação do software. Ao integrar a API do GitHub de forma assíncrona, elimina-se o acoplamento de dados fixos (*hardcoding*). O PortfolioHUB torna-se dinâmico: novos repositórios criados no perfil do GitHub passam a se refletir no portfólio de forma automatizada e transparente.

2.1. Criação e Configuração do Ambiente Dedicado

> **PROMPT DO ESTUDANTE:**
>
> *"Montei o escopo. Agora, como faço para configurar um ambiente dedicado no GitHub para o PortfolioHUB? Preciso da estrutura de pastas ideal para o projeto."*

RESPOSTA DO GEMINI:

Crie um repositório chamado `portfoliohub-core`. No seu ambiente de desenvolvimento, a estrutura de pastas deve seguir o padrão modular de desenvolvimento web moderno para assegurar manutenibilidade:

```
├── .github/
│   └── workflows/      # Automação de CI/CD (GitHub Actions)
├── src/
│   ├── assets/         # Recursos estáticos (imagens, ícones)
│   ├── components/     # Componentes visuais isolados e reutilizáveis
│   ├── services/       # Módulos de comunicação com a API do GitHub
│   └── utils/          # Funções utilitárias e formatadores de dados
├── .env.example      # Modelo de exibição das chaves de ambiente
├── .gitignore        # Restrição de arquivos locais para o Git
└── README.md         # Documentação técnica de arquitetura
```

2.2. Integração e Armazenamento Dinâmico de Projetos

> **PROMPT DO ESTUDANTE:**
>
> *"Como faço para que o PortfolioHUB busque meus repositórios e projetos do GitHub de maneira dinâmica, sem deixar dados fixos no código?"*

RESPOSTA DO GEMINI:

Utilize a API REST oficial do GitHub. No módulo `src/services/github.js`, implemente o consumo utilizando chamadas assíncronas (`async/await`). Exemplo de implementação limpa:

```
export async function fetchGitHubRepositories(username) {
    const token = process.env.GITHUB_TOKEN;
    try {
        const response = await fetch(`https://api.github.com/users/${username}/repos?
sort=updated`, {
            headers: {
                'Authorization': `token ${token}`,
                'Accept': 'application/vnd.github.v3+json'
            }
        });
        if (!response.ok) throw new Error('Falha ao obter dados do GitHub');
        const data = await response.json();
        return data.filter(repo => !repo.fork); // Retorna apenas projetos autorais
    } catch (error) {
        console.error("Erro na integração:", error);
        return [];
    }
}
```

3. Gestão de Usuários e Segurança

> **Resumo e Contextualização Técnica:** Esta seção abrange o estabelecimento de controle de acesso baseado em funções (RBAC) nas configurações do GitHub, bloqueio contra alterações diretas na branch de produção (*Branch Protection Rules*), proteção e ocultação de credenciais sensíveis através do arquivo `.gitignore` e ativação de escaneamento de vulnerabilidades através do GitHub Dependabot. Trata-se da aplicação real dos preceitos de **DevSecOps**. Isolar as credenciais em variáveis de ambiente impede um dos erros de segurança mais severos do mercado (o vazamento de tokens de acesso à nuvem), e as travas de branch garantem a integridade da aplicação estável.

3.1. Integração do Controle de Usuários com o GitHub

> **PROMPT DO ESTUDANTE:**
>
> *"O slide pede para configurar a gestão de usuários no GitHub e integrar ao PortfolioHUB. Como aplico o controle de acesso baseado em papéis (RBAC) e protejo minha branch principal?"*

RESPOSTA DO GEMINI:

A gestão de usuários baseia-se nas permissões do GitHub vinculadas à aplicação. Siga estas configurações de governança:

- **Configuração do RBAC:** No painel do repositório (*Settings > Collaborators*), atribua papéis estritos. Desenvolvedores juniores recebem permissão de `Read` ou `Write` limitadas, enquanto engenheiros de release retêm o papel de `Admin`.
- **Regras de Proteção de Branch (Branch Protection Rules):** Vá em *Settings > Branches* e adicione uma regra para a branch `main`. Ative obrigatoriamente:
  1. *Require a pull request before merging* (Exigir PR aprovado antes do merge).
  2. *Require status checks to pass before merging* (Garantir aprovação nos testes automáticos antes de consolidar o código).

3.2. Implementação de Políticas de Segurança DevSecOps

> **PROMPT DO ESTUDANTE:**
>
> *"Quais políticas de segurança robustas você me recomenda adotar para que meu projeto atinja nota máxima nos critérios de avaliação de segurança?"*

RESPOSTA DO GEMINI:

Com base nas melhores práticas de DevSecOps, o projeto deve implementar três defesas mandatórias:

1. **Isolamento Absoluto de Credenciais:** Nunca exponha o *Personal Access Token* (PAT) no código fonte. Declare-o no arquivo local `.env` e insira obrigatoriamente a linha `.env` no arquivo `.gitignore`.
2. **Princípio do Menor Privilégio:** Ao gerar o token de acesso no GitHub, configure os escopos (scopes) restritos ao mínimo necessário: ative apenas a flag `read:user` e `repo:status`. Bloqueie privilégios administrativos.
3. **Análise de Vulnerabilidade Ativa:** Ative o **GitHub Dependabot** nas configurações de segurança do repositório para realizar varreduras em tempo real nas dependências do `package.json`, mitigando ataques de cadeia de suprimentos (Supply Chain Attacks).

4. Compartilhamento e Controle de Acesso com GitHub

> **Resumo e Contextualização Técnica:** Esta fase gerencia a adoção técnica do fluxo de ramificação de funcionalidades (*Feature Branch Workflow*) e a padronização das mensagens de modificação através da convenção semântica *Conventional Commits*. O foco está na organização do ciclo de vida evolutivo do software e no trabalho em equipe. Isolar cada desenvolvimento em ramos secundários impede conflitos de código destrutivos, enquanto commits padronizados documentam o histórico técnico de forma auditável e clara, facilitando manutenções futuras.

4.1. Controle de Versão e Integração do Repositório

> **PROMPT DO ESTUDANTE:**
>
> *"Como deve ser o fluxo de trabalho diário utilizando o Git para garantir que o controle de versão funcione de forma profissional no PortfolioHUB?"*

RESPOSTA DO GEMINI:

Deve-se abolir o push direto na branch estável. Adote o fluxo de ramificação baseado em funcionalidades (Feature Branch Workflow):

1. Atualize sua base de dados local: `git checkout main && git pull`.
2. Crie um ramo isolado para a tarefa: `git checkout -b feature/integracao-api`.
3. Após codificar, adicione e consolide localmente as alterações.
4. Envie a branch para o servidor remoto: `git push origin feature/integracao-api`.
5. Abra um **Pull Request (PR)** na interface do GitHub para auditoria antes do merge final.

4.2. Documentação de Processos e Práticas de Colaboração

> **PROMPT DO ESTUDANTE:**
>
> *"Como documento os processos e garanto um padrão de colaboração limpo no histórico do Git do meu grupo ou projeto individual?"*

RESPOSTA DO GEMINI:

Implemente o padrão internacional de **Conventional Commits** para padronizar as mensagens de registro de modificações. As mensagens de commit devem seguir a seguinte estrutura semântica:

- `feat:` Quando uma nova funcionalidade for adicionada (ex: `feat: inclusao do card de tecnologias`).

- `fix:` Quando realizar a correção de uma falha ou bug (ex: `fix: tratamento de erro 404 da API`).
- `docs:` Quando houver alterações exclusivas na documentação (ex: `docs: atualizacao das instrucoes do README`).

Toda essa padronização e as regras de colaboração devem estar explicitadas claramente na introdução do arquivo `README.md` na raiz do projeto.

5. Finalização da Integração e Testes

> **Resumo e Contextualização Técnica:** Compreende a codificação dos tratamentos de exceção estruturados (como a mitigação de bloqueios por requisições excessivas da API - *Rate Limiting*) e a integração da automação de nuvem para Implantação Contínua (CD) conectada ao repositório central. O intuito é garantir robustez sistêmica e automatizar o deploy. A resiliência contra falhas de conexão (*Graceful Degradation*) garante a disponibilidade do portfólio via cache local caso a API caia, e o pipeline de CD garante que atualizações aceitas na branch estável reflitam na internet imediatamente, eliminando deploys manuais falhos.

5.1. Conclusão das Etapas de Integração via API

> **PROMPT DO ESTUDANTE:**
>
> *"Cheguei na fase final de integração. O que preciso verificar no código do PortfolioHUB para garantir que o consumo de dados está operando sem vazamentos ou gargalos?"*

RESPOSTA DO GEMINI:

Realize a validação do ciclo de requisições. Certifique-se de que o estado da aplicação gerencia corretamente o ciclo de vida dos dados assíncronos. Verifique o tratamento de erro estruturado (*Graceful Degradation*): caso o limite de requisições por hora da API do GitHub seja atingido (Rate Limiting), o PortfolioHUB deve exibir dados salvos em cache local (como o `localStorage`) e apresentar um alerta em tela, mitigando falhas críticas de quebra de interface (UI Crashes).

5.2. Preparação do Ambiente de Produção e Lançamento

> **PROMPT DO ESTUDANTE:**
>
> *"Como preparo a aplicação final para produção (deploy real) integrado ao GitHub para atender à entrega prática de ADS?"*

RESPOSTA DO GEMINI:

Para o lançamento em produção estável, execute as etapas de **Continuous Deployment (CD)**:

1. Conecte o repositório GitHub a um provedor Cloud Serverless (recomenda-se a **Vercel** ou **Netlify** pela automação nativa).
2. No painel de configurações da plataforma de nuvem escolhida, configure as **Environment Variables** (Variáveis de Ambiente) inserindo a chave de produção correspondente ao seu `GITHUB_TOKEN` de forma criptografada.
3. Valide o pipeline: realize um merge de teste na branch `main`. O servidor de hospedagem lerá o gatilho, executará o comando de compilação (build) automaticamente e disponibilizará a URL pública funcional para os avaliadores de ADS.

6. Revisão Final e Apresentação

> **Resumo e Contextualização Técnica:** É o fechamento e empacotamento do produto de software desenvolvido. Consiste na auditoria dos critérios da rubrica de avaliação e no desenho estratégico do roteiro técnico para o vídeo explicativo que será hospedado no YouTube. O objetivo desta etapa é a defesa de projeto. Ela consolida o conhecimento técnico de ADS, fornecendo ao estudante um guia estruturado de argumentação lógica para comprovar que as integrações automatizadas, as diretrizes DevSecOps estruturadas e as orientações geradas pela inteligência artificial foram aplicadas com sucesso.

6.1. Estruturação do Roteiro e Apresentação para YouTube

> **PROMPT DO ESTUDANTE:**
>
> *"A última seção exige um vídeo de apresentação no YouTube. Como devo estruturar o roteiro dessa gravação para demonstrar com clareza as soluções de ADS e os critérios superados?"*

RESPOSTA DO GEMINI:

Apresente um vídeo objetivo focado em demonstrações práticas e técnicas, estruturado no seguinte roteiro de 3 a 5 minutos:

- **Minuto 0:00 - 0:30 | Introdução Técnica:** Identificação pessoal (Arthur Bradley Barroca) e contextualização do objetivo do PortfolioHUB integrado ao ecossistema Git.
- **Minuto 0:31 - 1:45 | Demonstração da Aplicação:** Exibição do PortfolioHUB rodando em ambiente de produção na nuvem. Mostre o carregamento dinâmico e fluidez dos dados extraídos em tempo real do GitHub.
- **Minuto 1:46 - 3:00 | Defesa de Segurança e Arquitetura:** Abra o código-fonte e comprove a ocultação das chaves de ambiente com `.env` e `.gitignore`. Explique a configuração das regras de proteção de branch do repositório remoto.

- **Minuto 3:01 - Fim | Conclusão:** Breve consideração sobre como o suporte analítico do Google GEMINI auxiliou no cumprimento dos requisitos e aceleração do deploy DevSecOps.